

RAPPORT DE PROJET - JE1030PROJET

```
class Agent(threading.Thread):  
    def __init__(self, name, x, y):  
        """  
        name : nom pour id l'agent (str)  
        x, y : position de l'agent (float)  
        env : instance d'Environnement  
        """  
        threading.Thread.__init__(self)  
        self.name = name  
        self.x = float(x)  
        self.y = float(y)  
        self.env = env
```

PROJET INFORMATIQUE SMA “SYSTEME MULTI-AGENT”

PROJET RÉALISÉ PAR

ABBASSI YOUNES
ABDELAZIZ NAWELLE
BOUGUIMA KAWTHAR

PROJET ENCADRÉ PAR
PASQUET JÉRÔME



SOMMAIRE

I. Contexte général

II. Modélisation

2.1 L'architecture générale

2.2 Choix techniques

2.3 Améliorations

III. Gestion du projet

3.1 Répartition des tâches

3.2 Diagramme de Gantt

3.3 Détail semaine a semaine

IV. Conclusion

Interprétations des scénarios

Annexes et sources

CONTEXTE GÉNÉRAL

Ce projet s'inscrit dans le cadre du module de Systèmes Multi-Agents (SMA).

L'objectif est de concevoir un programme simulant le comportement d'une petite population d'individus dans un environnement 2D.

Chaque agent agit de manière autonome et interagit avec les autres, ce qui permet de faire émerger des comportements collectifs complexes à partir de règles simples. Le thème choisi est un village paisible inspiré du Japon, composé de maisons et d'un parc.

Les agents représentent des habitants qui se déplacent, se croisent, discutent, et peuvent être affectés par une maladie ou une rumeur.

Ces phénomènes permettent de simuler des effets de propagation (type effet papillon ou Fortembusch).

Le projet se découpe en deux parties :

- Une première phase en Python pour la logique et la simulation.
- Une seconde en Java pour l'interface graphique, reliée par un système de socket.

Scénario 1 - Trivial

Chaque agent choisit aléatoirement une destination dans l'environnement et s'y rend en évitant les obstacles.

S'il rencontre un autre agent, ils interagissent (transmission d'un état "malade").

Ce scénario montre le fonctionnement de base du SMA : perception locale + actions simples

Scénario 2 - Emergent

Un agent est initialement "malade" et se déplace dans l'environnement.

Les agents sains peuvent devenir malades après contact.

Aucun agent ne cherche volontairement à infecter ou éviter les autres la contagion est purement locale.

Pourtant, à grande échelle, on observe une diffusion épidémique collective

Scénario 3 - Emergent

Chaque agent a une humeur (heureux / neutre / triste).

Les agents "tristes" se rapprochent instinctivement d'autres agents pour améliorer leur humeur.

Les "heureux" ont tendance à s'isoler.

Aucune règle n'impose un regroupement global, mais localement, les interactions créent des zones de regroupement et d'isolement

MODÉLISATION

2.1 L'architecture générale

Notre système repose sur trois éléments principaux

Classe agent

Chaque agent possède un nom, une position (x, y) continue, un état de santé, une humeur et une destination

Classe environnement

Elle contient les lieux, obstacles et positions possibles pour les agents

Moteur de simulation

Il coordonne la création des agents, la mise à jour de leurs positions et leurs interactions

- Chaque agent se déplace dans un espace continu, ce qui permet de se rapprocher d'un comportement plus naturel que celui basé sur une grille. Leur trajectoire vers une destination est calculée grâce à un algorithme de pathfinding A* adapté à un espace continu (sans nœuds ni graphe).
- Chaque agent hérite de la classe Thread, ce qui lui permet d'avoir une autonomie d'exécution.
- Les agents se déplacent en direction de lieux choisis aléatoirement et peuvent interagir entre eux (propagation de maladie, changement d'humeur).
- L'environnement contient les obstacles et vérifie les collisions.

Les interactions modélisent un effet de type Fortembusch : une petite perturbation (un agent malade, une humeur) peut se propager et influencer progressivement tout le système.

MODÉLISATION

2.2 Choix techniques

CONCEPTION ET IMPLÉMENTATION DE L'ALGORITHME A*

1. INTRODUCTION ET RÔLE DANS LE PROJET

Dans le cadre de notre projet de simulation multi-agents (SMA), chaque membre de l'équipe est responsable d'un module indispensable au fonctionnement global du système. Pour ma part, j'ai conçu et implémenté l'algorithme A*, qui constitue le moteur de navigation des agents.

Ce module permet aux habitants de se déplacer dans la ville de manière optimale, d'éviter les obstacles (maisons, arbres, zones interdites), et d'atteindre leurs destinations (maison, parc, magasin, etc.) avec un comportement cohérent et réaliste.

Sans un système de pathfinding fiable, les agents seraient incapables de se déplacer correctement, ce qui rendrait la simulation inutilisable. L'A* joue un rôle central :

La modélisation définit la structure du monde, la gestion multithread gère la simultanéité des comportements, et l'algorithme de navigation permet à chaque agent de raisonner spatialement pour choisir le meilleur chemin.

Ainsi, ma partie fait le lien direct entre la vision abstraite du SMA (positions des bâtiments, obstacles, objectifs) et son exécution dynamique (déplacements fluides des habitants dans l'environnement).

2. OBJECTIFS DE MA CONTRIBUTION

L'objectif de mon travail était de doter les agents d'une véritable intelligence de déplacement, et non d'un simple mouvement aléatoire. Ils devaient être capables d'identifier les obstacles, d'évaluer plusieurs directions possibles, et de choisir un chemin raisonnable pour rejoindre un but donné, que ce soit une maison, un parc ou un commerce.

Au départ, je visais seulement un A* fonctionnant sur une grille. Mais très vite, les besoins du SMA m'ont amené à dépasser ce cadre discret : la ville n'est pas faite de « cases », les agents se déplacent dans un espace continu, et le réalisme visuel dépend fortement de la fluidité de leurs trajectoires. Mon objectif est donc progressivement devenu plus ambitieux : conserver la rigueur d'A*, tout en l'adaptant à un environnement continu, avec des positions flottantes et des obstacles représentés par des coordonnées réelles.

Un autre objectif important était de rendre le module suffisamment générique pour que les agents puissent réutiliser la même fonction dans des contextes variés, simplement en changeant le point de départ, le but, la liste d'obstacles et éventuellement leur mémoire interne.

2.2 Choix techniques

CONCEPTION ET IMPLÉMENTATION DE L'ALGORITHME A*

3. RAPPEL DU PRINCIPE DE L'ALGORITHME A*

Théoriquement, l'algorithme A* est fondé sur l'évaluation suivante :

$$f(n) = g(n) + h(n)$$

où

(g(n)) représente le coût déjà parcouru depuis le point de départ,

(h(n)) est une heuristique, c'est-à-dire une estimation du coût restant jusqu'à l'objectif,

(f(n)) est la somme des deux et sert à classer les positions à explorer.

L'algorithme maintient un ensemble de positions « ouvertes » dans une file de priorité (OPEN SET) et explore toujours en premier la position ayant la plus petite valeur de (f). Ce mécanisme lui permet d'équilibrer exploration réelle et anticipation, avec une bonne efficacité.

Dans les implémentations, la différence principale entre les versions provient du choix de l'heuristique et de la représentation de l'espace. Sur la grille, j'utilisais la distance de Manhattan. Dans la version finale `astar_final`, j'ai retenu la distance euclidienne, beaucoup plus naturelle dès que l'on travaille avec des coordonnées réelles.

4. DE LA VERSION DISCRÈTE À LA VERSION CONTINUE

Au début de mon travail, j'ai implémenté un A* classique sur une grille : la ville était représentée par une matrice 2D, les déplacements se faisaient case par case, et les obstacles étaient des cases marquées comme interdites. Cette étape m'a permis de bien comprendre les mécanismes fondamentaux d'A* : gestion des voisins, manipulation de la file de priorité `heapq`, utilisation d'un ensemble `visited` pour ne pas revisiter les mêmes positions, et retournement du chemin une fois l'objectif atteint.

Même si ce prototype fonctionnait, il restait trop limité pour notre SMA. Le monde de la simulation n'étant pas structuré par cases, il devenait artificiel de forcer les agents à évoluer sur une grille invisible. Le rendu visuel en pâtissait : trajectoires trop anguleuses, détours parfois peu naturels, et absence de notion d'orientation réelle.

C'est à partir de ce constat que j'ai décidé d'adapter A* à un environnement continu, dans lequel les positions sont exprimées en flottants, et où les obstacles ne sont plus des cases, mais des coordonnées dans un plan.

2.2 Choix techniques

CONCEPTION ET IMPLÉMENTATION DE L'ALGORITHME A*

5. PRÉSENTATION DE LA FONCTION ASTAR_FINAL

Voici la signature de ma fonction finale :

```
12 def astar_continu(start, goal, obstacles, step=0.5, obstacle_radius=0.6, agent_memoire=None):
```

Elle calcule un chemin optimal en espace continu à l'aide d'un A* adapté. L'espace est échantillonné autour de chaque position grâce à plusieurs directions possibles.

Les paramètres sont les suivants :

start et **goal** sont des tuples (x, y) représentant respectivement la position de départ et la position d'arrivée de l'agent.

obstacles est une liste de positions (x, y) correspondant aux obstacles fixes de l'environnement (par exemple les maisons, les arbres, ou d'autres zones où l'agent ne doit pas passer).

step définit la distance d'avancement à chaque itération, c'est-à-dire la longueur de chaque « pas » effectué par l'agent dans le plan.

obstacle_radius représente la distance minimale à respecter autour d'un obstacle pour considérer une position comme sûre.

agent_memoire est un ensemble de positions antérieurement visitées par l'agent, utilisé pour influencer le coût et éviter de repasser inutilement par les mêmes endroits.

La fonction renvoie une liste ordonnée de positions constituant le chemin calculé, ou une liste vide si aucun chemin n'est trouvé.

6. HEURISTIQUE ET DISTANCE EN ESPACE CONTINU

À l'intérieur de la fonction, j'ai défini une heuristique basée sur la distance euclidienne, via `math.hypot` :

```
32 # Distance Euclidienne
33 def heuristic(a, b):
34     return math.hypot(a[0] - b[0], a[1] - b[1])
```

Cette distance correspond à la longueur du segment droit joignant deux points. Elle est parfaitement adaptée à un espace continu, car elle représente réellement « l'effort » que l'agent devra fournir pour se rapprocher du but. Là où la grille imposait une distance de Manhattan, le passage au continu justifie naturellement la distance euclidienne, plus fidèle à la géométrie de l'environnement.

2.2 Choix techniques

CONCEPTION ET IMPLÉMENTATION DE L'ALGORITHME A*

7. GESTION DES OBSTACLES : ÉVOLUTION DE IS_FREE VERS ENV.IS_OBSTACLE

Dans une version intermédiaire de mon A*, j'avais conçu une fonction interne `is_free(x, y)` permettant de vérifier si une position candidate se trouvait suffisamment éloignée des obstacles. Cette fonction calculait la distance entre le point testé et chaque obstacle, et rejetait les positions trop proches en appliquant un rayon d'exclusion. Cette approche était pertinente tant que les obstacles étaient simplement représentés sous forme d'une liste de coordonnées.

Cependant, cette méthode a évolué lorsque la map finale de la simulation a été intégrée au projet. Mes camarades ont structuré l'environnement de manière plus complète, en centralisant la gestion des obstacles directement dans l'objet `env`. Cette nouvelle architecture inclut une fonction dédiée :

```
env.is_obstacle(x, y)
```

Cette fonction encapsule toute la logique de collision : elle sait si une position traverse un bâtiment, rencontre un mur, ou entre dans une zone interdite. Elle est utilisée par l'ensemble du SMA, ce qui garantit une cohérence totale entre les différents modules.

À partir de ce moment, ma fonction `is_free` est devenue redondante. Nous l'avons donc supprimée, et la ligne de code qui vérifiait auparavant la distance aux obstacles a été remplacée par :

```
if env.is_obstacle(nx, ny):  
    continue
```

Cette modification a été faite lors de la finalisation du module A*, et elle correspond à un choix naturel d'intégration : ce n'est plus l'algorithme lui-même qui évalue la proximité géométrique avec les obstacles, mais l'environnement complet de la simulation, basé sur la map construite par l'équipe.

Dans sa version finale, l'A* ne gère donc plus directement le rayon d'exclusion. Le contrôle des collisions est désormais centralisé dans l'environnement, ce qui rend le code plus léger, plus clair et mieux intégré au projet global.

8. VÉRIFICATION DES POSITIONS ET INTERACTION AVEC LA MAP

Dans la version finale de mon A*, la gestion des obstacles n'est plus assurée par une fonction interne comme auparavant. Initialement, j'utilisais une fonction `is_free(x, y)` qui mesurait la distance entre une position candidate et les obstacles afin d'empêcher l'agent de s'en approcher. Cette logique était adaptée aux prototypes mais ne correspondait plus à la structure finale du SMA.

En effet, une fois la map complète de la ville intégrée par mes camarades, les obstacles sont devenus des éléments directement portés par l'environnement global. Celui-ci possède sa propre méthode :

```
env.is_obstacle(x, y)
```

Cette méthode est désormais la référence commune à tout le projet : elle connaît précisément la forme, la taille et la position de chaque obstacle. Elle garantit que tous les modules (agents, affichage, collisions) travaillent sur la même information.

Ainsi, dans ma boucle principale, le test de collision est devenu :

```
if env.is_obstacle(nx, ny):  
    continue
```

Cela permet d'exclure les positions interdites sans que l'A* ait à gérer lui-même les distances ou la géométrie. Le module est plus simple, plus propre et parfaitement aligné avec la structure globale du SMA.

2.2 Choix techniques

CONCEPTION ET IMPLÉMENTATION DE L'ALGORITHME A*

9. ÉCHANTILLONNAGE DES DIRECTIONS EN CONTINU

L'un des points clés de ma version continue de l'A* est la génération, autour de chaque position courante, de 24 nouvelles positions possibles. Celles-ci correspondent à des orientations espacées de 15 degrés, ce qui permet au déplacement d'être fluide, non rectiligne, et beaucoup plus naturel qu'un mouvement limité à quatre directions.

La boucle suivante constitue le cœur de ce mécanisme :

```
58         # Espace continu (24 directions, tous les 15°)
59         for angle in range(0, 360, 15):
60             nx = current[0] + step * math.cos(math.radians(angle))
61             ny = current[1] + step * math.sin(math.radians(angle))
```

Une fois la position candidate calculée, elle est immédiatement testée via `env.is_obstacle(nx, ny)` afin de vérifier qu'elle est compatible avec la topologie réelle de la ville. Ce filtrage est essentiel pour maintenir une cohérence parfaite entre les mouvements de l'agent et les limites de la map.

Cette approche combine donc :

- la souplesse du déplacement continu,
- la variété des orientations possibles,
- et la précision fournie par la map environnementale.

Le résultat est une trajectoire qui donne l'impression que l'agent cherche véritablement son chemin dans un espace réaliste.

10. COÛT DU MOUVEMENT, MÉMOIRE INTERNE ET COMPORTEMENT INTELLIGENT

Le calcul du coût d'un mouvement reste un aspect crucial de l'algorithme. Dans ma version, chaque déplacement possède un coût de base proportionnel au pas `step`. Pour enrichir le comportement de l'agent, j'ai intégré une mémoire interne qui conserve les positions déjà visitées. Lorsque l'algorithme rencontre une position qui figure dans cette mémoire, le coût associé est augmenté.

```
67         # Coût du déplacement (pénalise les positions déjà connues)
68         step_cost = step * (1.5 if (nx, ny) in agent_memoire else 1.0)
69         # Coût total (g + h)
70         new_cost = cost + step_cost + heuristic((nx, ny), goal)
```

Cette pénalisation évite que l'agent repasse systématiquement par les mêmes endroits, ce qui rend sa navigation plus efficace et plus cohérente.

Une différence importante avec les versions précédentes est que désormais, la partie « obstacles » du coût ne dépend plus d'un calcul géométrique interne. L'A* ne cherche pas à mesurer la distance à un mur ou à un arbre, puisque cette responsabilité appartient désormais entièrement à l'environnement via `env.is_obstacle`. Cela permet de simplifier considérablement l'algorithme tout en maintenant un comportement réaliste.

La combinaison de l'heuristique euclidienne, du coût dynamique influencé par la mémoire, et de l'échantillonnage en continu assure un déplacement fluide, adaptatif et intelligent.

2.2 Choix techniques

11. RÉSULTAT DE LA FONCTION ET COMPORTEMENT DANS LA SIMULATION

Lorsque la condition d'arrêt est satisfaite, la fonction renvoie le chemin complet : `path + [goal]`. Ce chemin est une liste ordonnée de positions (x, y) représentant une trajectoire continue et fluide, non alignée sur une grille et adaptée à la géométrie réelle de la map. Grâce à la vérification `env.is_obstacle`, les positions retenues respectent automatiquement les contraintes de l'environnement simulé, puisque l'A* ne retient que des points explicitement autorisés par la map.

Dans la simulation, les agents n'utilisent généralement pas la totalité du chemin en une seule fois. Ils se déplacent d'un point à l'autre (souvent un seul pas à chaque mise à jour), puis recalculent si nécessaire un nouvel itinéraire en fonction de leur perception de l'environnement, des obstacles rencontrés ou d'un changement d'objectif. Cette manière de faire combine un calcul de chemin global avec une adaptation locale en continu, ce qui donne un comportement plus réactif et plus crédible.

Lorsque aucun chemin n'est possible, la fonction renvoie une liste vide. Cela peut se produire lorsque le but est entièrement enclavé derrière des obstacles déclarés dans la map ou si toutes les directions explorées sont interdites par `env.is_obstacle`. Dans ce cas, c'est au SMA de décider de la réaction de l'agent, par exemple en modifiant son objectif, en cherchant un autre point d'intérêt ou en adoptant un comportement temporaire d'attente.

12. CONCLUSION

La conception de mon algorithme A* a accompagné toute l'évolution du projet, depuis une première version sur grille jusqu'à une approche continue parfaitement intégrée à la map finale du SMA. Au fil de ces étapes, l'algorithme s'est adapté aux besoins réels de la simulation : passage aux coordonnées flottantes, exploration en vingt-quatre directions, ajout d'une mémoire interne pour éviter les retours inutiles, puis remplacement de l'ancienne détection d'obstacles par la fonction centralisée `env.is_obstacle`, mieux adaptée à la structure définitive de l'environnement.

La version finale de `astar_continu` s'insère naturellement dans le comportement des agents : elle produit des trajectoires fluides, cohérentes avec la géométrie de la ville, et permet une replanification continue en fonction des objectifs et de l'environnement.

Ce module constitue ainsi une pièce essentielle du SMA, assurant la cohérence entre la perception du monde, la décision de l'agent et son déplacement effectif. Il remplit désormais pleinement son rôle de moteur de navigation dans la simulation.

MODÉLISATION

2.2 Choix techniques

Multi-threading

Afin de rendre notre simulation plus crédible et fidèle au fonctionnement réel d'un système multi-agents, nous avons adopté une approche multithreadée : chaque agent est exécuté dans son propre thread Python, en héritant directement de la classe Thread.

Ainsi, chaque agent possède sa propre boucle de vie via la méthode `run()`.

Cette boucle s'exécute en continu et de manière concurrente, ce qui permet à chaque agent :

- de choisir une destination lorsqu'il n'a plus de chemin à suivre,
- de calculer et suivre progressivement un itinéraire,
- d'éviter dynamiquement les autres agents grâce à une fonction de vision locale (`eviter_agents()`),
- d'interagir avec les agents proches (propagation de maladie ou d'humeur),
- de déposer des phéromones selon son état émotionnel,
- tout cela indépendamment des autres agents.

Le multithreading reflète le parallélisme naturel des SMA : plusieurs agents prennent des décisions simultanément, sans attendre qu'un autre ait terminé son action.

Contrairement à une boucle séquentielle unique, cette architecture permet de simuler un comportement "temps réel", où les interactions, mouvements et adaptations se produisent en continu et de façon autonome.

Dans notre implémentation, la boucle `run()` coordonne toutes les actions de l'agent

Chaque agent évolue donc réellement en parallèle des autres, ce qui offre une dynamique plus fluide, plus réaliste et surtout émergente, c'est-à-dire que le comportement global du système naît naturellement des décisions locales indépendantes.

MODÉLISATION

2.3 Améliorations

Ajout des phéromones sociales

L'introduction de phéromones sociales permet de renforcer le comportement collectif des agents.

Inspiré du modèle des fourmis (Voir Annexes), chaque agent laisserait une "trace sociale" influençant les décisions des autres (direction, humeur, regroupement, etc.).

Le choix de déplacement serait alors probabiliste, selon la formule :

$$p = a \times p_1 + (1 - a) \times p_2,$$

où p_1 est la probabilité de direction sans phéromone et p_2 dépend de la concentration locale de phéromones.

Cela permettrait de faire émerger des comportements de groupe complexes, comme la formation de foules ou la diffusion d'émotions collectives.

Affichage avec carte

Une amélioration majeure serait de remplacer l'affichage texte par une interface graphique basée sur une carte visuelle (map)

Cela permettrait de visualiser en temps réel les déplacements et interactions des agents, rendant la simulation plus immersive et plus lisible.

Affichage avec carte

Une amélioration majeure serait de remplacer l'affichage texte par une interface graphique basée sur une carte visuelle (map)

Cela permettrait de visualiser en temps réel les déplacements et interactions des agents, rendant la simulation plus immersive et plus lisible.

MODÉLISATION

2.3 Améliorations

Sockets

On a utilisé le concept de sockets pour permettre la communication entre la partie modélisation (python) et la partie affichage (java). Dans notre projet, on a créé une socket côté Python pour envoyer les données de la modélisation, et une socket côté Java pour recevoir ces données et les afficher. On établit une connexion entre les deux, puis on envoie les informations sous forme de flux, ce qui permet à notre interface Java de visualiser en temps réel les résultats calculés par Python.

Deamon thread

On a ajouté des daemon's threads pour gérer la communication via les sockets de manière asynchrone. Cela permet à notre programme Java de continuer à fonctionner normalement, même pendant que les données sont reçues depuis Python. On a choisi des daemon's threads démons parce qu'ils s'arrêtent automatiquement quand le programme principal se termine, ce qui évite que notre application reste bloquée ou continue de tourner inutilement en arrière-plan. Ces threads sont donc pratiques pour gérer des tâches de communication continues sans bloquer le reste de l'application.

GESTION DU PROJET

3.1 Répartition des tâches

Nawelle Abdelaziz

Développement complet du module de pathfinding : conception de plusieurs versions successives de l'algorithme (grille, nœuds, puis espace continu) et implémentation finale de l'A adapté au déplacement continu dans la simulation.

Younes Abbassi

Modification A* en adaptant le bon pas et un système de débogage ,réalisation des Sockets,création d'un slider de vitesse pour accélérer la simulation,mise en place d'une mémoire pour les agents

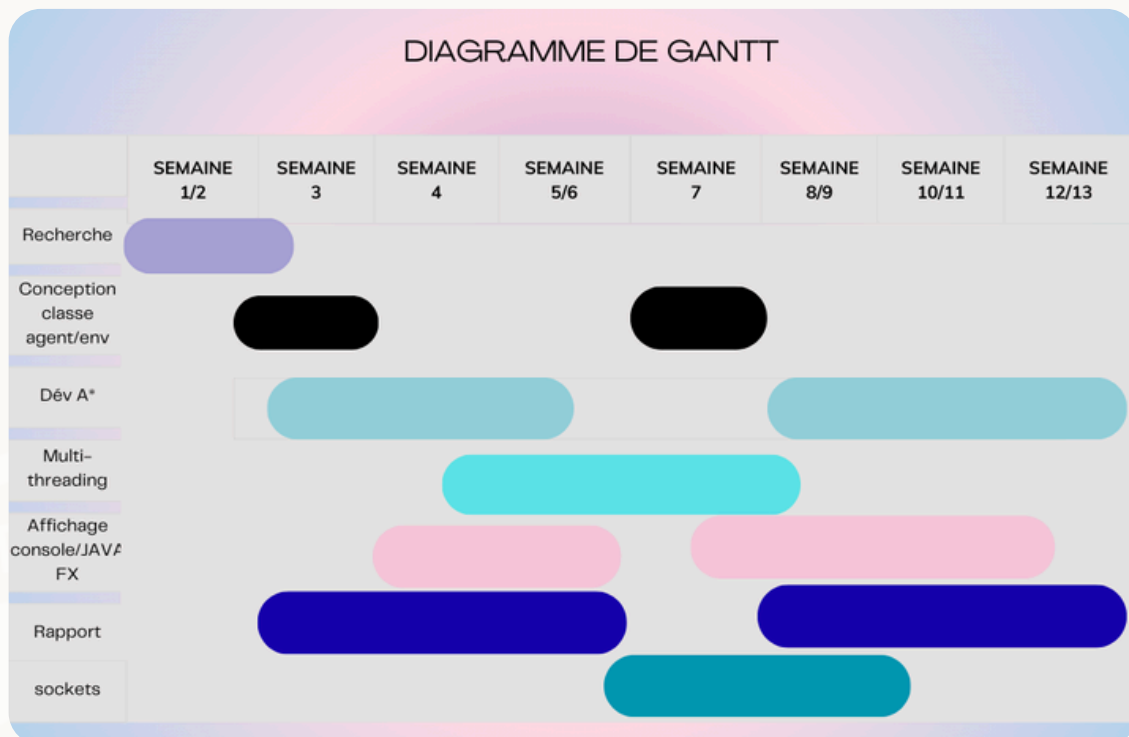
Bouguima Kawthar

Modélisation de l'environnement, logique de simulation, gestion des déplacements et intégration des threads puis affichage en JavaFX et ajout des obstacles a partir de la map

Travail commun

Tests, mise au point, rédaction du rapport

3.2 Diagramme de Gantt



GESTION DU PROJET

C.f ReadMe

Nawelle Abdelaziz

- S1-2 : Étude de l'algorithme A*, adaptation à un espace continu sans nœuds
- S3 : Implémentation du déplacement progressif des agents à l'aide de A*
- S4-5 : Synchronisation avec l'environnement
- S6 : Création du diagramme
- S7: Correction du multithread
- S8-11: Amélioration du A*, ajout de l'exploration et de la mémoire,...
- S11-13: Rédaction de la partie A* du rapport

Kawthar Bouguima

- S1-S2 : Étude du concept SMA, création de la classe Environnement, ajout des obstacles et lieux
- S3 : Intégration de la propagation maladie / rumeur et gestion des interactions
- S4-S5 : Tests de cohérence, affichage console et amélioration de la boucle de simulation
- S6 : Création de la diapo et test finaux du premier rendu
- S7 : Renseignement sur l'affichage en JavaFX
- S8 : Création du menu et fenêtre de simulation
- S11 : Ajout de la vision de la classe Agent pour gérer les collisions
- S12/13 : Dernière amélioration, test finaux

Abbassi Younes

- S1/S6 : Autre groupe (Conception A*, rapport, diaporama et diagramme de Gantt)
- S7/S8 : Sockets
- S8 : Modification du A*
- S9 : Slider bouton de vitesse
- S10 : Remplissage rapport
- S11 : Ajout de la mémoire
- S12/13 : Vitesse, rédaction rapport, amélioration des sockets, rajout diagramme de Gantt

MÉTRIQUES

Interprétations des scénarios

Scénario routine

On pourrait mesurer le taux de propagation de l'état "malade" : c'est-à-dire la proportion d'agents infectés au fil du temps.

Cette métrique permet de mesurer directement l'effet des interactions simples entre agents dans l'environnement et de vérifier que la transmission se produit comme prévu.

Scénario de contagion locale

On peut mesurer le nombre moyen d'agents infectés par agent initialement malade, c'est-à-dire la proportion d'agents qui deviennent malades à partir d'un agent infecté au départ. Cela permet de visualiser comment la contagion se diffuse à grande échelle, même si chaque agent n'interagit qu'avec ses voisins.

Scénario d'humeur

On peut mesurer l'indice de regroupement local, c'est-à-dire la densité d'agents autour de chaque agent selon son humeur. Cela permet de visualiser comment les comportements individuels créent spontanément des zones de regroupement ou d'isolement dans l'environnement.

CONCLUSION

Ce projet nous a permis de comprendre concrètement le fonctionnement d'un système multi-agents et les défis liés à la synchronisation d'entités autonomes.

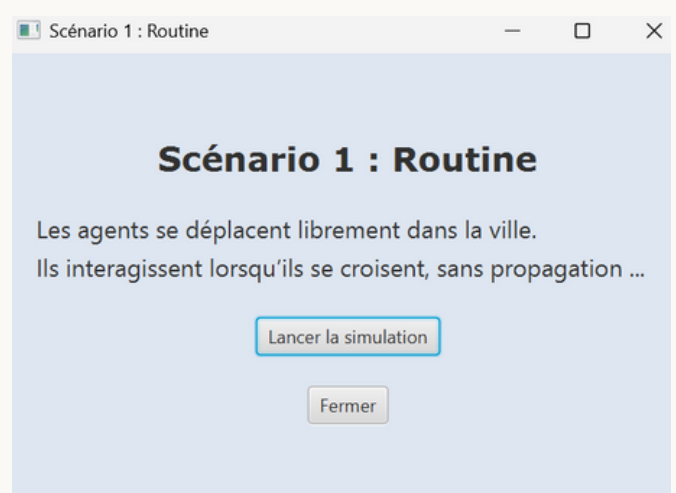
L'utilisation du multithreading nous a offert une approche plus réaliste et dynamique des interactions, tandis que l'adaptation de A* en espace continu a permis d'obtenir un comportement fluide sans recourir à une grille discrète.

Nous avons ainsi intégrés une interface graphique en Java, a l'aide de la bibliothèque graphique JavaFX afin de visualiser la simulation en temps réels

Menu de la simulation



Fenêtre explicative



Bonus

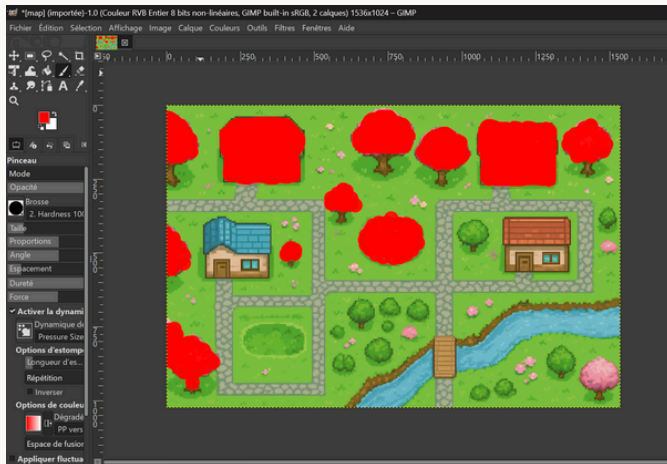


ANNEXES

Outils externes utilisé :

Gimp :

Dessin des obstacles de la map



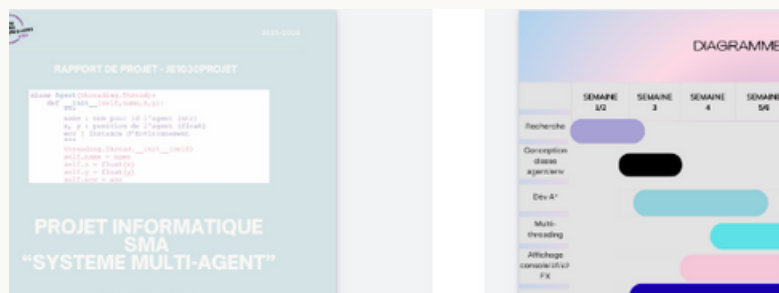
JavaFX :

Bibliothèque graphique de Java

```
--module-path
"C:\JavaFX\lib"
--add-modules
javafx.controls,javafx.fxml
--enable-native-access=javafx.graphics
```

Canva :

Création du rapport , slides et création des boutons du menu



Pillow :

Librairie d'imageries python

```
PS C:\Users\kawy\Desktop\PROJET_SMA_GP7> pip install Pillow
Collecting Pillow
  Downloading pillow-12.0.0-cp310-cp310-win_amd64.whl (7.0 MB)
    7.0/7.0 MB 20.4 MB/s eta 0:00:00
Installing collected packages: Pillow
Successfully installed Pillow-12.0.0

[notice] A new release of pip is available: 23.0.1 -> 25.3
[notice] To update, run: C:\Users\kawy\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.10_qbz5n2kfr
```

SOURCES

Idées phéromones sociales :

<https://home.mis.u-picardie.fr/~yli/docs/VT/VTProjet.pdf>

Lecture sur A* :

<https://www.datacamp.com/tutorial/a-star-algorithm>

Lecture sur la conception des agents :

https://turing.cs.pub.ro/auf2/html/chapters/chapter2/chapter_2_1_2.html

Lecture sur les système multi agent avec propagation de maladie :

<https://bmcpublichealth.biomedcentral.com/articles/10.1186/s12889-024-18157-x>

Conception des sockets

<https://pypi.org/project/jpysocket/>

Lecture multithreading :

<https://medium.com/@fedihamdi.jr/multithreading-en-python-3374f6b7dbd7>

Aide apportée par l'assistant IA (ChatGPT)

- la mise en place correcte du multithreading en structurant chaque agent comme un thread indépendant,
- la simplification et adaptation du A* en espace continu, sans recourir à une structure de nœuds,
- Corrections des fautes d'orthographe docstring
- Grande partie de l'affichage en JavaFX
- Explication des bug afin d'y remédier